



Discussion 1

C++ Review & Classes

UC Berkeley Master of Molecular Science
& Software Engineering

Constructors & Destructors

- C++ classes, like python classes, run *constructors* when objects of that class are initially created
 - What is the equivalent in Python?
 - How can C++ constructors be different than those in python?
- C++ classes can also have *destructors*
 - Kind of like Python?

Constructors & Destructors

- If not specified, compilers will generate some for you
- If you specify one, the compiler may not generate the others

```
class Circle
{
    private:
        double radius_;
};
```

```
int main(void) {
    // compiler generated Circle()
    Circle c;

    // compiler generated copy constructor
    Circle c2(c);

    return 0;
}
```

Constructors & Destructors

- If not specified, compilers will generate some for you
- If you specify one, the compiler may not generate the others

```
class Circle
{
    private:
        double radius_;

    public:
        Circle(double r) : radius_(r) {}

};
```

```
int main(void) {
    // ERROR - no Circle() constructor
    Circle c;

    // ERROR - no copy constructor
    Circle c2(c);

    return 0;
}
```

“default”-ed functions

- If not specified, compilers will generate some for you
- Can force compiler to generate one that would otherwise not be

```
class Circle
{
    private:
        double radius_;

    public:
        Circle(double r) : radius_(r) {}

        Circle(const Circle & other) = default;
};
```

```
int main(void) {
    // Still not ok
    Circle c;

    // Ok now
    Circle c2(c);

    return 0;
}
```

“default”-ed functions

- If not specified, compilers will generate some for you
- Can force compiler to generate one that would otherwise not be

```
class Circle
{
    private:
        double radius_;

    public:
        Circle() = default;
        Circle(double r) : radius_(r) {}

        Circle(const Circle & other) = default;
};
```

```
int main(void) {
    // Ok now
    Circle c;

    // Ok now
    Circle c2(c);

    return 0;
}
```

Rule of 3

- If not specified, compilers will generate some for you
- If you specify one, the compiler may not generate the others
- “If a class requires a user-defined copy constructor, destructor, or copy assignment operator, it very likely requires all three.”

Miscellaneous – “Most vexing parse”

```
int main(void) {  
    // Ok  
    Circle c(4);  
  
    // Warning??  
    Circle c2();  
  
    return 0;  
}
```

- C++ compilers can interpret the second version as a function declaration

```
x.cpp:16:13: warning: empty parentheses interpreted as a function declaration [-Wvexing-parse]  
   16 |     Circle c();  
       |             ^~  
x.cpp:16:13: note: replace parentheses with an initializer to declare a variable  
   16 |     Circle c();  
       |             ^~  
       |             {}  
1 warning generated.  
  
x.cpp:16:13: warning: empty parentheses interpreted as a function declaration [-Wvexing-parse]  
   16 |     Circle c();  
       |             ^~  
x.cpp:16:13: note: replace parentheses with an initializer to declare a variable  
   16 |     Circle c();  
       |             ^~  
       |             {}  
1 warning generated.
```

Miscellaneous – “Most vexing parse”

```
int main(void) {  
    // Ok  
    Circle c(4);  
  
    // Ok now  
    Circle c2{};  
  
    // or leave off the parentheses  
    Circle c3;  
  
    return 0;  
}
```

https://en.wikipedia.org/wiki/Most_vexing_parse